数据分类

1. 实验目的

通过神经网络程序实现数据分类问题；

2. 实验数据

具有一定随机性的散点分布数据。

3. 实验环境

Nvidia Jetson TX1 嵌入式系统；

4. 准备工作

4.1 导入必要的模块

- torch 模块，构造网络
- matplotlib 模块，可视化显示结果

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
```

4.2 构造样本集

- 实验用分类数据的形式

两类数据点。第一类是正态分布的 100 个随机点，均值为 2，方差为 1；第二类是正态分布的 100 个随机点，均值为-2，方差为 1。

- 相关代码

torch.normal(mean, std)

返回一个张量（Tensor），张量的随机数是从相互独立的正态分布中随机生成的；产生符合均值为 mean、标准差为 std 的正态分布的随机数；mean 和 std 可以是一个数组，两个数组的大小形状必须相同；为数组时，生成的 Tensor 变量的大小与数组大小相同。

✧ 一维数组（参见基本问题）

创建一维 Tensor 类型数组 x，生成随机值张量 y，其符合均值为 1、标准差为 1 的正态分布。

```
x = torch.ones(5)
y = torch.normal(x, 1)
```

✧ 二维数组

如果把 x 改成二维数组，例如：

```
x = torch.ones(2, 2)
y = torch.normal(x, 1)
```

● 实验用数据生成

构建两种不同的样本集（两类数据点），两个不同的散点集符合不同的正态分布。

```
data = torch.ones(100, 2)
x0 = torch.normal(2*data, 1)          # 第一类数据
x1 = torch.normal(-2*data, 1)         # 第二类数据
```

✧ x0 和 x1 都继承了 data 的大小，是一个二维数组，大小为 100×2；

✧ x0 和 x1 都包括了 x 坐标和 y 坐标，即第一个维度存储 x 坐标，第二个维度存储 y 坐标，每一个 x 坐标和 y 坐标都符合正态分布；

✧ x0 和 x1 分别表示一个点集，因为正态分布的参数（均值）不同，所以是两种不同的点集，绘制的散点图如图 3.1 所示（某次运行结果）。

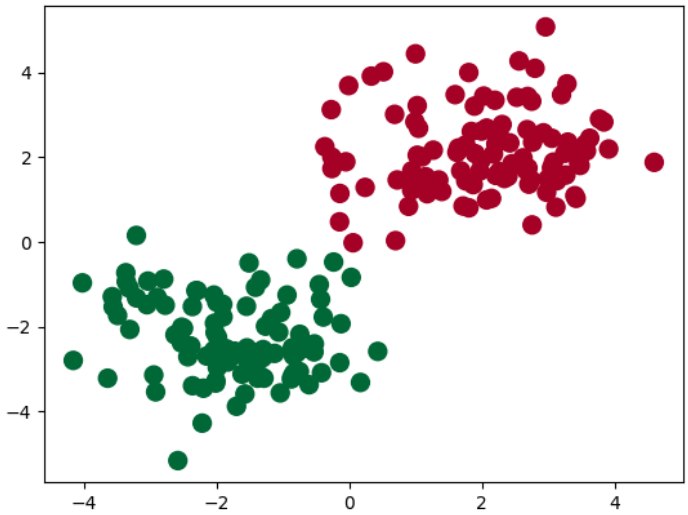

图 3.1 符合不同分布的散点集

✧ 数据合并

在网络训练时，希望训练集一次性输入网络，所以将 x0 和 x1 合并成一个样本集，合并 Tensor 张量的接口如下：

torch.cat((张量 1, 张量 2), 维度)

张量 1 和张量 2 为两个将要合并的张量；维度参数为 0 或 1，0 代表按列合并，1 代表按行合并，默认情况是按列合并。

```
x0 = torch.ones(2, 2)
x1 = torch.zeros(2, 2)          # 创建二维 Tensor 张量
x = torch.cat((x0, x1), 0)      # 按列合并
x = torch.cat((x0, x1), 1)      # 按行合并
x = torch.cat((x0, x1))         # 默认按列合并
print(x)
```

输出结果如下：

```
tensor([[1., 1.],\n [1., 1.],\n [0., 0.],\n [0., 0.]])
```

本实验将样本按列合并，这样合并之后的第一列代表所有的 x 坐标，第二列代表所有的 y 坐标。

```
# x0、x1 合并成 x，并转换成 Float 类型的 Tensor 变量
x = torch.cat((x0, x1), 0).type(torch.FloatTensor)
```

✧ 标签生成

给散点集 x0、x1 打上标签，第一类标记为 0，第二类标记为 1，用 y 表示。

```
y0 = torch.zeros(100)           # 第一类标签设置为 0
y1 = torch.ones(100)            # 第二类标签设置为 1
# y0、y1 合并成 y，并转换成 Long 类型的 Tensor 变量
y = torch.cat((y0, y1)).type(torch.LongTensor)
```

由于一个标签对应一对坐标，因此 y 只需要一个维度。在遇到分类问题时，PyTorch 中要求标签为 Long 类型的 Tensor 变量，这里了使用 type()接口将 y 转换为 LongTensor 变量。

5. 构建网络

本实验是一个分类问题，网络需要两个输入（x、y 坐标）和两个输出（概率

值）。所构建网络隐层节点数为 15，网络为三层结构，激活函数为 ReLU，网络结构如图 3.1 所示。隐含层激活函数选择 ReLU，Softmax 函数输出结果。

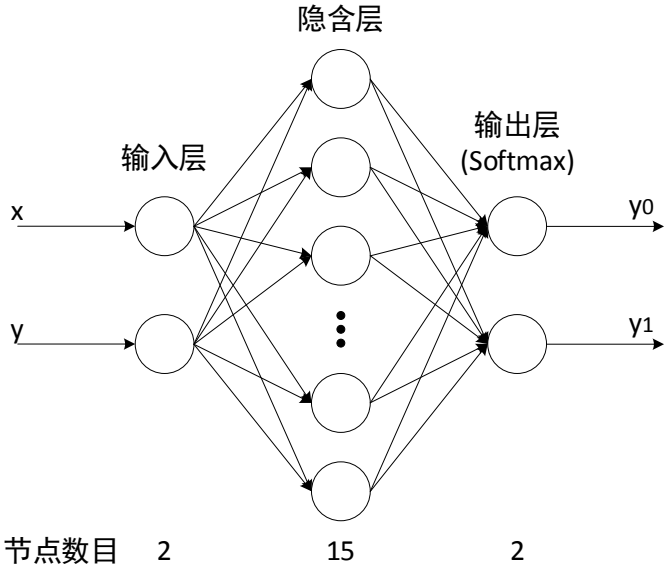

图 3.1 网络结构

构建该网络的方法和回归问题的构建方法类似。

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.classify = nn.Sequential(
            nn.Linear(2, 15),
            nn.ReLU(),
            nn.Linear(15, 2),
            nn.Softmax(dim=1)
        )
    def forward(self, x):
        classification = self.classify(x)
        return classification
```

5.1 训练网络

网络训练采用 SGD 算法进行优化。因为是分类问题，所以采用交叉熵函数作为损失函数，接口为 CrossEntropyLoss()。

```
net = Net()
optimizer = torch.optim.SGD(net.parameters(), lr=0.03)
loss_func = nn.CrossEntropyLoss()
for epoch in range(100):
    out = net(x)
    loss = loss_func(out, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

5.2 可视化

✧ 最大值及下标获取

为直观显示分类效果，采用不同的颜色对数据点进行标记，要达到这个效果，需要对网络的输出进行一定的处理。

```
torch.max()  #该接口用来求一个 Tensor 张量中的最大值
x = torch.rand(2, 2)
m = torch.max(x)          # 返回张量的最大值（四个值中的最大一个）
# 返回每一列的最大值及其下标，torch.return_types.max(values=tensor([*, *]), \n indices=tensor([*, *]))
m = torch.max(x, 0)
# 返回每一行的最大值及其下标，torch.return_types.max(values=tensor([*, *]), \n indices=tensor([*, *]))
m = torch.max(x, 1)
m = torch.max(x, 0)[0]    # 只返回每一列的最大值
m = torch.max(x, 1)[0]    # 只返回每一行的最大值
m = torch.max(x, 0)[1]    # 只返回每一列的最大值下标
m = torch.max(x, 1)[1]    # 只返回每一行的最大值下标
```

网络通过 Softmax 输出有两个值，输出结果为 100 行 2 列，每一行代表一个分类结果，保留每一行的最大值的下标。如果下标值为 0，则这一行中第一个值

较大，热键编码 10，分为第一类；如果下标值为 1，则这一行中第二个值较大，热键编码 01，分为第二类。

```
classification = torch.max(out, 1)[1]          # 返回每一行中最大值的下标
# 为便于计算准确率，需要把下标（也表示标签）转换成 numpy 数组
class_y = classification.data.numpy()
# 为便于计算准确率，需要把标签转换成 numpy 数组
target_y = y.data.numpy()
```

✧ 绘制散点图

```
plt.scatter(x.data.numpy()[:, 0], x.data.numpy()[:, 1], c=class_y, s=100, cmap = 'RdYlGn')
```

- x：第一列作为横坐标，第二列作为纵坐标；
- c=class_y：颜色随着 class_y 的值的不同而不同；
- 颜色模式采用 RdYlGn，也可以不设置颜色模式，使用默认颜色。

✧ 准确率

```
accuracy = sum(class_y == target_y) / 200 # 计算准确率
# 显示准确率
plt.text(1.5, -4, f'Accuracy={accuracy}', fontdict={'size': 20, 'color': 'red'})
```

6. 实验任务

同学自己编写完整的数据分类的训练程序和测试程序，并显示输出结果。同学可以尝试不同的层数、隐层节点数、改进或自行设计网络模型，完成实验任务。